

Demo Script — Agentic AI Hackathon

A step-by-step recording guide for a ~4–5 minute demo video, built around the AI review loop rather than the rule-based reporting underneath it.

The story this demo tells: a deterministic rule engine scores every overdue invoice cheaply and unattended. An AI agent then re-examines those decisions with context the rules structurally cannot see — the customer's payment history and total exposure — and either backs the rules, escalates beyond them, or argues against them. A human approves everything. The interesting moments are the disagreements.

Setup — first time on a new machine

If you're starting from a fresh clone rather than an existing bench, do this first. `<site>` and `<port>` below are whatever `install.sh` printed for you — it defaults to `receivables-risk.local` on port `8080`, but confirm from your own terminal output rather than assuming.

```
curl -fsSL https://raw.githubusercontent.com/phivinhkien1710-sudo/receivable_risk_manager/v1.1.0/install.s
```

This installs the app with **no data** — the dataset is a public, non-PII benchmark, but it's gitignored for repo-size reasons, so it isn't part of the clone. Get it from this release's assets:

```
# Download demo_500_open.csv from:
# https://github.com/phivinhkien1710-sudo/receivable_risk_manager/releases/tag/v1.1.0
# Save it to:
apps/receivable_risk_manager/local_data/demo_500_open.csv
```

Then load it in — same command as the reset step below, just for the first time:

```
./apps/receivable_risk_manager/scripts/demo_reset.sh <site>
```

One dependency `install.sh` cannot set up for you: the Claude CLI needs to be logged in and reachable by whatever machine runs the background worker, under whatever subscription this demo should bill to. Run `claude login` before attempting an AI Review Run — see the checklist below for how to confirm it's actually reachable, not just installed.

Before you hit record

Environment checklist (all of these matter, each has bitten this app for real):

```
# 1. Web server up
curl -sI http://<site>:<port> | head -1          # expect: HTTP/1.1 200 OK

# 2. Exactly ONE worker running -- a duplicate worker caused a 20x slowdown
ps aux | grep "bench_helper frappe worker" | grep -v grep
# If two appear, kill the non-launched one. Two workers competing made an
# AI review take 20+ minutes per chunk instead of ~60 seconds.

# 3. Claude CLI reachable BY THE WORKER (not just your shell)
bench --site <site> console <<'EOF'
from receivable_risk_manager.services.ai_review import claude_cli_available
print("CLI reachable:", claude_cli_available())
EOF
```

Reset to a clean demo dataset (~7 seconds — safe to re-run anytime, including right before recording, to guarantee a clean state):

```
cd /path/to/frappe-bench
./apps/receivable_risk_manager/scripts/demo_reset.sh <site>
```

This wipes existing receivables data and re-imports 498 open invoices through the real `Receivables Import Job` path, so batch tracking is populated and the Batch Workflow menu has something to attach to.

Login: `http://<site>:<port>` — Administrator (password is in `~/passwords.txt` after a fresh `install.sh` run)

Recording tip: the AI review itself takes roughly **60–90 seconds** for a 20-row run. That's real dead air. Either cut it in post, timelapse it, or use the waiting time for the voiceover section explaining what the agent is doing.

Scene 1 — The problem (~20s)

Navigate: `Receivables Invoice list`

4  **Capture:** the invoice list showing 498 rows.

Point out: every row is an overdue invoice. A human cannot read 498 of these and decide what to chase first. Neither can they read 48,000, which is what the full dataset holds.

Scene 2 — Import with batch tracking (~30s)

Navigate: `Receivables Import Job` → open the most recent completed job

4  **Capture:** the completed job showing row counts (498 valid rows imported).

Point out: the import is batch-tracked — every invoice it brought in is recorded as a member of this specific batch, so every later stage can be scoped to "just this import" instead of the whole database.

4  **Capture:** click the **Batch Workflow** menu to show the dropdown: `Run AI Review`, `Review Proposals`, `Disagreements Only`, `AI Review History`.

Point out: this menu is the spine of the workflow. Each entry opens a pre-filled form or a scoped queue — nothing starts a stage by itself.

Scene 3 — Launching the agent (~30s)

Navigate: `Batch Workflow` → `Run AI Review`

4  **Capture:** the new AI Review Run form with **Import Batch** pre-filled.

Set `Limit Rows` **to 20** — keeps the demo tight and the run around 60–90s.

Save, then click **Start Review**.

4  **Capture:** the confirmation dialog, which states plainly that nothing is sent to any customer and every proposal still needs approval.

Point out: an AI Review Run is a *visible unit of work*. It's started deliberately by a human, scoped to a batch, and it records exactly what it looked at and what it decided. Nothing about the agent's judgment happens invisibly inside a cron job.

Scene 4 — What the agent actually sees (~40s, over the waiting time)

While the run executes, explain the mechanism. Optionally show the prompt file on screen:

receivable_risk_manager/services/prompts/invoice_review_instructions.md

 **Capture (optional):** the system prompt, scrolled to the "How to judge" section.

Point out: the agent is never shown a blank slate. Every invoice it reviews carries the rule engine's own recommendation as `rule_recommended_action`. It's not asked "what should happen here" — it's asked "here's what the rules already decided; does the fuller picture change that?"

The fuller picture is the part rules can't see: - **Pattern vs. anomaly** — a customer who always pays 42 days late being 45 days late is behaving normally. A customer who always pays on time being 45 days late is a real signal. - **Concentration of exposure** — one invoice may be small, but the customer may owe millions across dozens of other open invoices.

Scene 5 — The results (~40s)

Navigate: the AI Review Run, once status reads `Completed`

 **Capture:** the Results section counters.

Real numbers from an actual run (`AIRV-00013` , 20 rows, 69 seconds):

Counter	Value	Meaning
Reviewed	20	assessments the agent examined
Proposed Action	20	some action warranted
Actions Created	5	genuinely new Collection Actions
Already Had An Action	15	existing action re-annotated, not duplicated
Agreed With Rules	11	agent backed the rule engine
Disagreed With Rules	9	the rows worth a human's time

Point out: the run cannot claim work it didn't do. "Proposed 20" and "created 5" are deliberately separate numbers — an earlier version reported only the former and was effectively lying about its own output.

 **Capture:** the orange headline alert — "*Claude disagreed with the rule engine on 9 of 20 invoices reviewed.*"

Scene 6 — The disagreements (the payoff, ~60s)

Navigate: click **Review Disagreements** on the run

 **Capture:** Collection Action Queue, filtered to this run's disagreements.

 **Capture:** the columns side by side — **Rule Said, AI Said, Agreed, AI Confidence, AI Reasoning.**

Open one disagreement. Real examples from actual runs:

Agent escalating beyond the rules — "*COSTCO normally pays only 4.1 days late but is 74 days overdue — a dramatic departure from pattern. Customer has \$2.29M open exposure.*" The rules saw one overdue invoice; the agent saw a reliable customer behaving abnormally at scale.

Agent arguing for less — on BJ'S us , 66 days overdue: "*a large, normally reliable customer, 1.4-day average delay... too anomalous for routine escalation; investigate.*" It proposed `Immediate Follow-up` instead of the rules' `Escalate Collection`.

Point out — this is the safety story, say it explicitly:

The agent may escalate above the rule baseline on its own authority, and that proposal is used directly. But when it argues for something *less* severe, the rule's action stays in force — the disagreement is recorded for a human to adjudicate, not applied. An LLM never silently talks this system out of chasing a debt. Worst case, a human spends ten seconds rejecting a proposal.

Two more gates worth naming: - Below a confidence threshold, the agent's verdict is discarded entirely and the rules decide alone. - A malformed or unrecognized response sorts as "No Action," so it can never outrank the rules by accident.

Scene 7 — Human in the loop (~30s)

Navigate: open a `Proposed` Collection Action

  **Capture:** the workflow buttons — **Approve** / **Reject**.

Click Approve.

  **Capture:** status transitioning `Proposed` → `Open` .

Point out: nothing the agent proposes takes effect until a human with the Accounts Manager role approves it. The agent's entire output is proposals.

Scene 8 — The audit trail (~20s)

Navigate: `Risk Audit Log` , filter by the invoice you just approved

  **Capture:** entries tagged `Source = AI Agent` alongside `Pipeline` , `Invoice Risk` , `Customer Risk` .

Point out: every scoring decision, every agent verdict, and every disagreement leaves an entry. You can reconstruct why the system concluded anything, months later.

Closing frame (~20s)

What runs unattended: customer aggregation, risk scoring, invoice assessment, and stale-action escalation — all deterministic, all cheap.

What a human deliberately starts: every piece of AI judgment.

What is never automated: contacting a customer. The app produces proposals; a person decides.

Screenshot inventory

Capture these; they're the ones that carry the story on their own:

#	Screen	Why it matters
1	Invoice list, 498 rows	the scale problem
2	Completed import job	batch tracking exists
3	Batch Workflow menu open	the workflow spine
4	AI Review Run form, batch pre-filled	deliberate human trigger
5	Confirmation dialog	nothing is sent, approval required
6	Results counters	honest self-reporting
7	Disagreement headline alert	the hook
8	Queue: Rule Said / AI Said / Reasoning	the core product surface
9	One disagreement, full reasoning	the agent's actual judgment
10	Approve → Open transition	human in the loop
11	Risk Audit Log, Source = AI Agent	explainability

Existing screenshots in `screenshots/` (`collection-action-queue.png` , `customer-risk-overview.png` , `invoice-collection-priority.png` , `receivables-customer-list.png` , `receivables-invoice-list.png`) predate the AI review feature and show the older rule-only flow — recapture rather than reuse for anything AI-related.